Lab 12

Graphical User Interface using JavaFX (Part 2)

Objectives: ***At the end of this lab session the student should be able to:***

- Bind objects to UI components.
- Work with files through GUI.

Introduction:

1. Data binding allows synchronization between the user interface components and the underlying data model.
2. Data binding ensures that changes in the data model are automatically reflected in the user interface.
3. Reading and writing files are combined with JavaFX for a user-friendly interaction experience.

Problem Solving Exercises

Exercise 1: Country Explorer

- Create a new JavaFX project and a countryEx package.
- Import to Eclipse the provided Country.java class, CountryRepository.java class, and countries.dat file.
- Implement the following to develop the Country View App.

1. Choose TableView<Country> as the root layout.
2. Resize the Scene to be 800 pixels in width and 400 pixels in height.
3. Set the window title (Stage) as 'Country App'.
4. Create columns for each attribute in the Country class, as follows:

```
TableColumn<Country, String> codeColumn = new TableColumn<>("Code");
codeColumn.setCellValueFactory(new PropertyValueFactory("code"));
```

5. Add created columns to the tableView using the tableView.getColumns().addAll() method.
6. Retrieve country data from 'countries.dat' file by calling the LoadCountries() method in the CountryRespository class.
7. Convert the loaded list of country data into an ObservableList using the following method:

```
ObservableList<Country> data = FXCollections.observableList(CountryRespository.countries);
```

8. Set the data as the content of the tableView
9. Create a ComboBox for the continent and another one for the region.
10. Get the continents list by calling CountryRespository.getContinents()

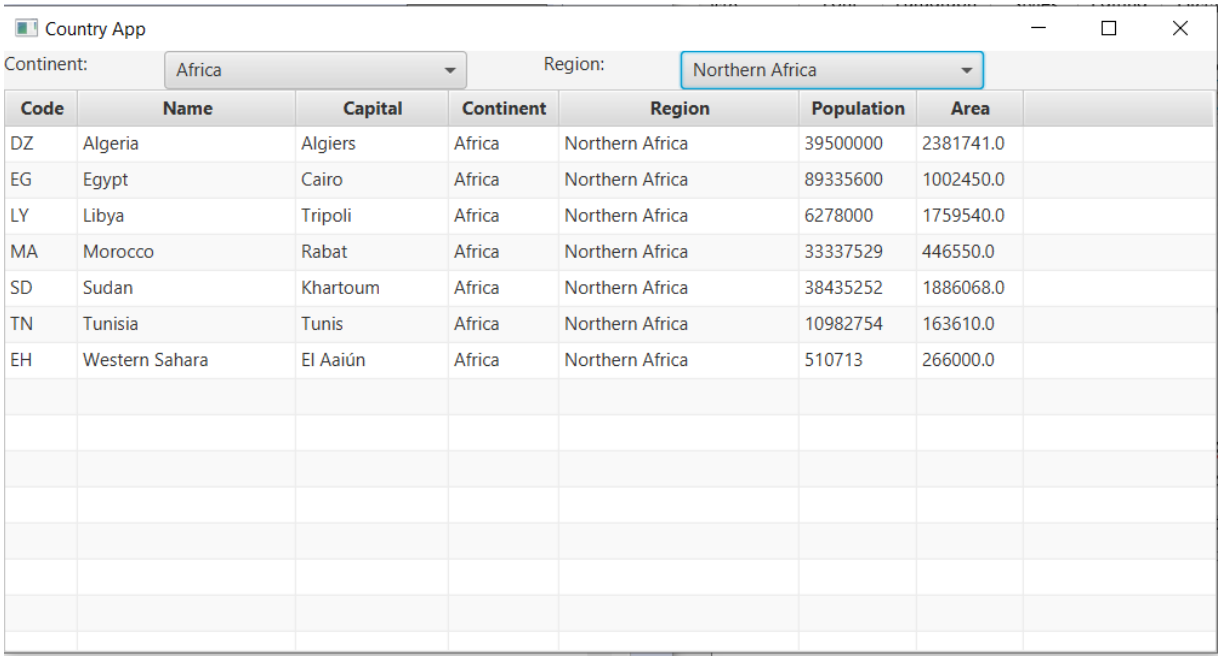
11. Convert the continents list into an observableList, then set it as the items of the comboBox, as follows:

```
ComboBox<String> continentComboBox = new ComboBox<>(); ObservableList<String>
continentData = FXCollections.observableList(CountryRespository.getContinents());
continentComboBox.setItems(continentData);
```

12. Set the event handler for the continentComboBox to do the following:
    a. Get the selected continent from the continentComboBox
    b. Get the regions of the selected content by calling the CountryRespository.getRegions(selectedContinen)
    c. Convert the regions list into ObservableList then pass it to regionComboBox using setItems() method
13. Use '200' as the width of the created ComboBoxes using the setPrefWidth(value) method.
14. Place the ComboBoxes with their labels at the top of the window using the following code:

```
HBox hBox = new HBox();
hBox.getChildren().addAll(new Label("Continent"),continentComboBox,
new Label("Region"), regionComboBox); hBox.setSpacing(50);
Scene scene = new Scene(new VBox(hBox, root), 800, 400); primaryStage.setScene(scene);
```

15. Set the event handler for the regionComboBox to do the following:
    a. Get the selected region from the regiontComboBox
    b. Get the countries of the selected region by calling the CountryRespository.getCountriesByRegion(selectedRegion)
    c. Convert the countries list into ObservableList then pass it to the tableView using setItems() method
16. Run the CountryApp to test that the countries in the table are re-filled whenever a region is selected.



Exercise 2: Trips Register

- Create a new package named tripEx.
- Import to Eclipse the Trips.java class and trips.dat file.
- Implement the following to develop the Trip Register App that allows users to keep track of their business and leisure trips.

1. Use TableView<Trip> as the root layout.
2. Resize the Scene to be 400 pixels in width and 400 pixels in height.
3. Set the window title (Stage) as 'Trips App'.
4. Create columns for each attribute in the Trip class.
5. Add created columns to the tableView.
6. List in the tableView the trips returned by calling the TripRepository.getTrips() method, as follows:

```
ObservableList<Trip> tripsdata =
FXCollections.observableArrayList(TripRepository.getTrips()); root.setItems(tripsdata);
```

7. Create two buttons, one to add a trip and the other to delete a trip.
8. Set the On Action event for the buttons to do the following:

I. The Add Trip button:

a. Create a new Scene that uses Grid Pane Layout
b. Resize the Scene to be 350 pixels in width and 250 pixels in height.
c. Create a new Stage and set its title to 'Add a Trip'.
d. Use the show() method to display the newly created Stage.
e. Add all required controls (5 labels, 2 radio buttons, 2 date Picker, 2 combo boxes, and 2 buttons). Use the getCountries() and getRatings() methods to fill the countryComboBox and the ratingComboBox. The following piece of code shows how to create a place the two radio buttons:

```
// Create two radio button for the trip type RadioButton radioButton1 =
new RadioButton("Business"); RadioButton radioButton2 = new
RadioButton("Leisure");

 // Put them in a toggle group
ToggleGroup toggleGroup = new ToggleGroup();
radioButton1.setToggleGroup(toggleGroup);
radioButton2.setToggleGroup(toggleGroup);

// Create a HBox layout to hold the radio buttons HBox radioHBox =
new HBox(10); radioHBox.getChildren().addAll(radioButton1,
radioButton2);
```

f. Add all controls to the grid pane layout (new created Scene)
g. Use setVgap(10), setHgap(20), and setPadding(new Insets(20)) for a better clean look.
h. Add event handler for the created buttons. The 'Ok' button should obtain the trip data from the UIs, create a new object of type Trip, add this newly created trip object to the tableView, and then closes the newly created Stage. The following snippet shows the created event handler for the okBtn:

```
//event handler for the okay button
                okBtn.setOnAction(e -> {
  String country =
countryComboBox.getSelectionModel().getSelectedItem(); String type =
                        ((RadioButton)
toggleGroup.getSelectedToggle()).getText();
LocalDate from = fromDatePicker.getValue(); LocalDate to =
toDatePicker.getValue(); String rating =
ratingComboBox.getSelectionModel().getSelectedItem(); Trip t = new
Trip(country,type,from,to,rating); tripsdata.add(t);
stage2.close();
                });
```

II. The Delete Trip button:

a. Get the selected trip object from the tableView using root.getSelectionModel().getSelectedItem()

b. Remove it from the trips arraylist returned in step 6, as follows: tripsdata.remove(selectedTrip)

